Proof of Random Leader: A Fast and Manipulation-Resistant Proof-of-Authority Consensus Algorithm for Permissioned Blockchains Using Verifiable Random Function

Md. Mainul Islam, Mpyana Mwamba Merlec, and Hoh Peter IN

Abstract—Proof of Authority (PoA) is a widely adopted consensus algorithm for permissioned blockchain networks, where a group of trusted entities governs the network. PoA is known for achieving rapid consensus with minimal computational and energy requirements. However, existing PoA variants such as Aura and Clique suffer from low transaction throughput in high workload conditions and provide limited randomness in leader selection. They are also vulnerable to time and order manipulation attacks. To overcome these limitations, this paper introduces a novel PoA-based consensus algorithm called Proof of Random Leader (PoRL), which utilizes a verifiable random function to enhance transaction throughput, improve scalability, and ensure fair and unpredictable leader selection. The proposed PoRL algorithm was implemented in Python and evaluated using a network of six consensus nodes with varying computational capabilities. The performance of PoRL was assessed based on key metrics, including security, consistency, availability, fault tolerance, block time, and transaction throughput. Experimental results indicate that PoRL achieves lower consensus times and higher transaction throughput compared to Aura and Clique, making it a more efficient solution for permissioned blockchain networks. The findings of this study provide valuable insights for blockchain practitioners in selecting the most suitable PoA implementation based on their specific network requirements.

Index Terms—Permissioned blockchain, consensus algorithm, proof of authority (PoA), proof of random leader (PoRL), verifiable random function (VRF), Aura consensus, Clique consensus, time manipulation attacks.

1 Introduction

Consensus is the process by which distributed nodes in a decentralized network agree on the final state of data. In blockchain systems, consensus algorithms are essential for maintaining data consistency and ensuring all nodes retain the same view of the distributed ledger [1]–[3]. They play a critical role in guaranteeing the integrity, security, and trustworthiness of data [4]. Moreover, these algorithms enable fault tolerance, allowing blockchain networks to continue operating even if certain nodes fail or go offline. This fault tolerance ensures seamless transaction verification and enhances the reliability of the network [5], [6]. As the number of nodes in the network grows, achieving consensus becomes increasingly challenging [7]. To address these challenges, efficient consensus algorithms are required to achieve higher transaction throughput and shorter block processing times without compromising security [8].

Blockchain itself is an immutable distributed ledger that records transactions in tamper-proof, time-stamped blocks, which are chronologically connected [1]–[4]. Managed through a peer-to-peer network, blockchain relies on consensus nodes that adhere to strict block creation and validation protocols [5], [6]. Blockchain networks can be categorized into permissioned and permissionless types based on access authorization [3]–[5]. Permissioned blockchains require prior authorization for participation, while permissionless blockchains allow anyone to join the network.

Additionally, blockchain networks can be classified into three governance models: public, private, and consortium [3]–[5]. Public blockchains are open to everyone, enabling any participant with the necessary software and a copy of the ledger to take part in decision-making as a miner or validator. Private blockchains, in contrast, restrict access to a small group of users or members within an organization. Consortium blockchains combine the advantages of both public and private networks, allowing collaborative governance by a selected group of entities.

Over the past decade, blockchain consensus algorithms have evolved significantly to address diverse network requirements and application scenarios [1]–[3]. In the literature, there are numerous consensus algorithms, but the most popular algorithms are proof of work (PoW), proof of stake (PoS), byzantine fault tolerance (BFT), and proof of authority (PoA) [1]–[6]. Each algorithm offers unique trade-offs regarding computational complexity, energy consumption, speed, and security [7], [8].

PoW is widely used in permissionless blockchains, such as Bitcoin, due to its robustness against Sybil attacks and

- *M. M. Islam is with the Department of Electrical and Computer Engineering, Texas A&M University, College Station, 77843, USA (e-mail: mainul.islam@ieee.org)*
- *M. M. Merlec is with the Department of Computer Science and Engineering, Korea University, Seoul 02841, South Korea (e-mail: mlecjm@korea.ac.kr).*
- *H. P. IN is with the Department of Computer Science and Engineering, Korea University, Seoul 02841, South Korea, and also with DAO Solution Inc., Seoul 06247, South Korea (e-mail: hoh_in@korea.ac.kr).*

Manuscript received XXXX; revised XXXX.
(Corresponding author: Hoh Peter IN.)

its ability to ensure decentralized security through computational puzzles [1]–[6]. However, the high energy consumption and resource requirements make it impractical for permissioned blockchains [7]–[9]. PoS, on the other hand, addresses the energy inefficiency of PoW by assigning block creation rights based on the stakes of participants [1]–[6]. Although PoS is more energy efficient, it has the disadvantage of potentially leading to wealth concentration, as validators with higher stakes have a higher probability of being selected for block proposals, raising concerns about network centralization. This centralization risk limits the use of PoS in permissioned blockchains, which often require a balance between fairness and efficiency [7], [8].

In permissioned blockchain networks, BFT and PoA consensus mechanisms are widely adopted due to their low energy consumption and ability to achieve high transaction throughput [8]–[13]. BFT-based algorithms, such as PBFT [15] and Istanbul BFT (IBFT) [16], ensure consistency and fault tolerance in the presence of a limited number of malicious nodes. They do not rely on the real-world identity or reputation of nodes. Instead, they achieve consensus through cryptographic techniques and protocol rules, tolerating Byzantine faults where nodes may behave maliciously. This flexibility enables them to function in both permissioned and permissionless environments. However, their scalability is constrained by a quadratic increase in communication as the number of nodes grows, driven by the numerous messages exchanged per step. This results in higher bandwidth requirements and significant network overhead, with a memory complexity of $O(n^2)$ [17]–[21].

In contrast, PoA measures the authority of a node based on real-world reputation, ensuring reliability and accountability. Validators are pre-approved, and their actions are tied to their real-world identities, which discourages malicious behavior because their reputation is at stake. PoA achieves lower latency and higher scalability than BFT by utilizing a simplified communication model. With a computational complexity of $O(n)$, compared to $O(n^2)$ for BFT-based algorithms, it is particularly suitable for permissioned blockchain networks, where the emphasis is on efficiency among known participants [17]–[26].

PoA also differs from BFT-type protocols in its fault tolerance capabilities. It can tolerate up to $f < N/2$ faulty nodes in a network with N nodes, provided that the majority of nodes remain honest [11]–[13]. This threshold includes both crash and malicious faults (Byzantine behavior), as long as the network has more honest validators than faulty ones. However, unlike PBFT, which can tolerate up to $f < N/3$ Byzantine faults while maintaining consistency in adversarial environments, PoA does not achieve full BFT [13]. If more than half of the validators become malicious due to external cyberattacks, PoA systems cannot maintain consensus, risking data integrity. This limitation highlights the trade-off in PoA between efficiency and security when compared to more communication-intensive BFT protocols [17]–[20]. While PoA lacks the robustness of BFT in adversarial scenarios, it can operate effectively through mechanisms where blocks are independently verified by multiple validators, and validators found to be acting maliciously are penalized and potentially removed from the network through voting [11]–[13].

Despite its advantages in efficiency and simplicity, PoA faces critical vulnerabilities related to fairness and security. Studies have shown that existing PoA algorithms, such as Aura and Clique, are susceptible to time and order manipulation attacks due to their deterministic leader rotation and block proposal mechanisms [27], [28]. Malicious actors can exploit these features to manipulate timestamps or delay block generation, disrupting transaction and block order while compromising the stability of the consensus mechanism [29], [30]. To address these vulnerabilities, introducing randomness into the leader selection process is essential. Integrating a Verifiable Random Function (VRF) into this mechanism offers unpredictability, ensuring that the process of selecting leaders remains unbiased. This enhancement strengthens the security of PoA systems, mitigating risks of collusion and manipulation while preserving their operational efficiency.

This paper proposes a novel algorithm, Proof of Random Leader (PoRL), which incorporates a VRF into the leader selection process to mitigate time and order manipulation attacks while improving PoA's throughput under high workloads.

1.1 Related Work

To address vulnerabilities including time and order manipulation attacks and the centralization risks inherent in traditional PoA algorithms such as Aura and Clique, recent studies have proposed various modifications and enhancements. Alrubei *et al.* [25] proposed HDPOA, an honesty-based distributed PoA consensus protocol that enhances the security and scalability of traditional PoA, making it suitable for resource-constrained IoT environments. Wu *et al*. [26] proposed DyPoA, an enhanced PoA protocol that dynamically adjusts the set of validators based on node behavior and network conditions, thus improving scalability and fault tolerance. Zhang *et al.* [27] explored vulnerabilities in Aura and proposed mechanisms to counter time manipulation attacks, which exploit the deterministic nature of leader selection. The predictability of leader selection in traditional PoA protocols can be exploited for order manipulation attacks, as demonstrated in [28]. Similarly, in [29] they have proposed mechanisms to mitigate front-running attacks in Clique, where malicious actors exploit the predictability of block proposer selection to manipulate transaction order, undermining the fairness of the network. The attack of clones against PoA is described in [30], where multiple malicious validators are created to take control of a PoA network, exposing the risks of relying on a small set of trusted validators and highlighting the need for more resilient consensus mechanisms.

A promising effective solution to these issues is to introduce randomness in leader selection through the use of VRFs [31]–[39]. VRFs take a series of inputs to produce a random output along with cryptographic proof of authenticity that can be publicly verified, ensuring transparency and security [31], [32]. Cao *et al.* [32] highlighted the potential of VRFs in achieving fair leader election, while Guo *et al.* [33] optimized VRF-based leader selection to reduce computational overhead, making it more practical for real-world applications. Moreover, techniques for embedding tamper-resistant and publicly verifiable random number seeds in

blockchain systems have been proposed in [34], contributing to the enhancement of randomness generation and security in consensus protocols.

Beyond PoA, VRFs have been applied to a range of consensus mechanisms to enhance fairness and prevent manipulation. For instance, Werapun *et al*. [35] proposed NativeVRF for Ethereum Virtual Machine (EVM)-based blockchains to enhance the security and reliability of random number generation. Wang *et al* [36] developed a verifiable multiparty random number generator to ensure fairness, verifiability, and tamper resistance in delegated PoS consensus systems. A non-interactive VRF node extraction approach was introduced in [37], where the unpredictability and verifiability of node identities are achieved by making it impossible to identify key nodes before the block proposal. In addition, a reputation model was proposed to dynamically update the node status and select nodes with greater credibility to construct a group of consensus nodes.

Li *et al.* [38] extended PBFT with VRFs to adapt to dynamic network environments, demonstrating the versatility of VRFs in improving blockchain consensus. Yao *et al.* [39] introduced a practical anonymous VRF for blockchain applications, which improves the consensus algorithm of hyperledger fabric. In this approach, the correctness of the function value from which the output is computed can be anonymously verified. Their proposed VRF ensures anonymity, consistency, uniqueness, and pseudo-randomness. Opposed to them, we learn consensus nodes as identified rather than anonymous in permissioned blockchains.

While existing VRF-based mechanisms focus primarily on delegated PoS or BFT, this paper proposes PoRL, a VRF-based PoA algorithm tailored for permissioned blockchains. By incorporating VRFs, PoRL achieves unbiased and random leader selection, addressing the predictability and low throughput issues inherent in traditional PoA protocols such as Aura and Clique. Our work contributes to the ongoing research by presenting a novel PoA-based consensus mechanism that significantly improves the fairness, scalability, and security of permissioned blockchain networks.

1.2 Our Contributions

The contributions of this paper are summarized as follows:

1) *Attack Resistance*: We propose PoRL, a novel permissioned blockchain network consensus protocol that leverages VRFs to achieve unbiased and random leader selection. The proposed algorithm mitigates the predictability and manipulation vulnerabilities present in existing PoA algorithms such as Aura and Clique, ensuring fairness and enhancing security in permissioned blockchain networks.
2) *Enhanced Throughput and Latency*: By optimizing the leader selection process and eliminating the second communication round required in Aura, PoRL significantly reduces block time and increases transaction throughput, making it more suitable for high-load scenarios.
3) *Evaluation and Comparison*: We implemented and tested the proposed PoRL algorithm in a local area network setting using six consensus nodes with varying computational power. The performance of PoRL was evaluated in terms of security, consistency, availability, fault tolerance, block time, and transaction throughput, demonstrating lower consensus time and higher throughput compared to existing PoA protocols such as Aura and Clique. The results indicate that PoRL provides a balanced trade-off between security and performance, making it a suitable choice for a wide range of permissioned blockchain applications.
4) *Guidelines for Practical Implementation*: This research offers valuable insights for permissioned blockchain practitioners, providing guidelines on the selection of consensus protocols based on network requirements. By presenting a theoretical analysis and practical implementation of PoRL, this study bridges the gap between theory and practice in the design and deployment of PoA-based consensus protocols.

The rest of this paper is organized as follows: Section 2 elaborates on the background of this research study. Section 3 presents the proposed PoRL algorithm. Section 4 analyzes the fault tolerance property of PoRL. Section 5 provides a security model for PoRL. Section 6 analyzes the security of PoRL. Section 7 provides a comparative analysis of the experimental results and network performance. Section 8 mentions the limitations and scope of this research. Finally, Section 9 concludes the paper.

2 Background

2.1 Aura and Clique

Aura and Clique are two widely used PoA variants integrated with Ethereum clients that were originally proposed as a component of the Ethereum private blockchain ecosystem [11], [13], [24], [40]. The consensus procedure for each of these algorithms varies significantly, but they all start with the current term leader proposing a new block. A round-robin schedule is used to reach the agreement, with each authority becoming a leader in turn [11]–[13].

Fig. 1a shows the Aura algorithm's message patterns [12], [13]. Where in the first round, only the current leader proposes a block. The followers validate the block in the second round. When the majority of authorities have given their approval, the block is stored in the blockchain. If a malicious block containing invalid transactions is broadcast by the block proposer, the block is rejected, and the honest majority of the network votes the proposer out.

The message patterns of the Clique algorithm are illustrated in Fig. 1b [12], [13]. Clique, in contrast to Aura, permits multiple block proposers at each step. The block number and number of authorities are taken into consideration when choosing the proposers in a specified sequence. The number of messages per step and latency are minimized because there is no block acceptance sharing round. Blocks are proposed in a step by $N/2 - 1$ authorities including a leader. Thus, an authority can propose a block after every $N/2+1$ steps. As each follower is restricted from submitting a block for a random amount of time, the leader's blocks usually reach all of the followers first [12]. *Forks* can occur when multiple authorities submit their blocks simultaneously.

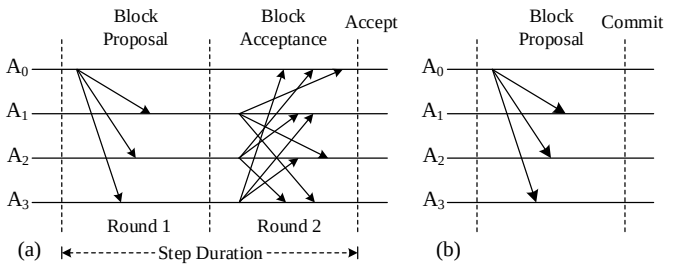

Fig. 1. Message patterns of PoA algorithms: (a) Aura and (b) Clique [13].

A *fork* is resolved using the GHOST protocol [41], [42] or a block scoring approach in which one of the blocks proposed by several authorities is accepted at a step, with the leader's block receiving the highest score.

2.2 Elliptic Curve Cryptography

The elliptic curve digital signature algorithm (ECDSA) [43] is used to sign and verify transactions. An elliptic curve over a prime field $\mathbb{F}_p$ is provided by the following equation [44]:

$$E_{a,b} : y^2 = x^3 + ax + b \tag{1}$$

where p is a prime number, $x, y, a, b \in [1, p-1]$, and

$$4a^3 + 27b^2 \neq 0 \tag{2}$$

Secp256k1 is a version of $E_{a,b}$ widely used in blockchains. For this curve, $a = 0$, $b = 7$, and $p = 2^{256} - 2^{32} - 977$ [45]. Therefore, the secp256k1 curve can be expressed as follows:

$$E : y^2 = x^3 + 7 \tag{3}$$

The group of points $E(\mathbb{F}_p)$ is a finite, additive, cyclic group of order q and generator point P.

A public key Q is a point on E, which is generated by the elliptic curve point multiplication (ECPM) operation. The underlying principle of ECPM is to multiply the base point $P \in E(\mathbb{F}_p)$ with an integer or private key $k \in \mathbb{F}_p$ such that $Q = kP \in E(\mathbb{F}_p)$. ECPM includes operations of a group of elliptic curves such as point addition and point doubling [46]. Consider two points on E in affine coordinates, such as $P(x_1, y_1)$ and $Q(x_2, y_2)$, in the range of $[1, p-1]$. The point addition $P + Q$ makes another point $R(x_3, y_3) \in E(\mathbb{F}_p)$, which is obtained as follows [44]:

$$\begin{aligned} R(x_3, y_3) &= \{P(x_1, y_1) + Q(x_2, y_2)\} \in E(\mathbb{F}_p) \\ x_3 &= \lambda^2 - x_1 - x_2 \\ y_3 &= \lambda(x_1 - x_3) - y_1 \end{aligned} \tag{4}$$

where $\lambda = (y_2 - y_1)(x_2 - x_1)^{-1}$ and $P \neq Q$.

The point doubling of $P(x_1, y_1)$ on E is performed as follows [44]:

$$\begin{aligned} R(x_3, y_3) &= 2P(x_1, y_1) \in E(\mathbb{F}_p) \\ x_3 &= \lambda^2 - 2x_1 \\ y_3 &= \lambda(x_1 - x_3) - y_1 \end{aligned} \tag{5}$$

where $\lambda = (3x_1^2)(2y_1)^{-1}$ and $P = Q$.

Algorithm 1 Binary Method for ECPM [46]

Curve parameter: Base point P
Input: Private key $k = \sum_{i=0}^{l-1} k[i]2^i; k[i] \in \{0, 1\}, k[l-1] = 1$
Output: Public key Q

```
P ← P(X = x, Y = y, Z = 1)        // affine to projective
Q ← P
for i from l − 2 to 0 do
    Q ← 2Q                          // PD
    if k[i] = 1 then
        Q ← Q + P                   // PA
    end if
end for
Q ← Q(x = X/Z^2, y = Y/Z^3)       // projective to affine
return Q
```

Point addition and doubling in affine coordinates involve several division operations over $\mathbb{F}_p$ [47] that are considerably time-consuming because a single modular division is equivalent to 80 modular multiplications in terms of latency [44]. To reduce ECPM time by avoiding modular division operations, the group operations are performed in projective coordinates by representing the points P and Q as $(X_1 = x_1, Y_1 = y_1, Z_1 = 1)$ and $(X_2 = x_2, Y_2 = y_2, Z_2 = 1)$, respectively. The formula for projective point addition is given by the equation [44]:

$$\begin{aligned} R(X_3, Y_3, Z_3) &= P(X_1, Y_1, Z_1) + Q(X_2, Y_2, Z_2) \in E(\mathbb{F}_p) \\ X_3 &= \alpha^2 - \beta^3 - 2X_1 Z_2^2 \beta^2 \\ Y_3 &= \alpha(X_1 Z_2^2 \beta^2 - X_3) - Y_1 Z_2^3 \beta^3 \\ Z_3 &= Z_1 Z_2 \beta \end{aligned} \tag{6}$$

where $\alpha = Y_2 Z_1^3 - Y_1 Z_2^3$ and $\beta = X_2 Z_1^2 - X_1 Z_2^2$.

The formula for projective point doubling is given by the equation [44]:

$$\begin{aligned} R(X_3, Y_3, Z_3) &= 2P(X_1, Y_1, Z_1) \in E(\mathbb{F}_p) \\ X_3 &= 9X_1^4 - 8X_1 Y_1^2 \\ Y_3 &= 3X_1^2(4X_1 Y_1^2 - X_3) - 8Y_1^4 \\ Z_3 &= 2Y_1 Z_1 \end{aligned} \tag{7}$$

ECPM can be performed by adding P to itself $k-1$ times such that:

$$Q = P + \underbrace{P + ... + P}_{k-1 \textbf{ times}} \tag{8}$$

If k can be expressible as a power of 2, Q can be computed by doubling P on itself $log_2 k$ times such that:

$$Q = \underbrace{...2(2(2(P)))}_{log_2 k \textbf{ times}} \tag{9}$$

Algorithm 1 presents the binary method for public key generation from a private key. The public key Q is computed by the sequential operations of point addition and point doubling based on the binary bit pattern of l-bit k. Point doubling is performed in every iteration. However, point addition is performed only when the current bit of k is 1.

Definition 1: Given an elliptic curve E defined over a finite field $\mathbb{F}_p$, a base point $P \in E(\mathbb{F}_p)$ of order q, and a point $Q \in \langle P \rangle$. Determine the integer $k \in [0, q-1]$ such that $Q = kP$. The integer k is called the discrete logarithm of Q to P, which is denoted as $k = log_P Q$. It is difficult to

Algorithm 2 ECDSA Signature Generation [43]

Curve parameter: Base point P, curve order q
Input: Private key k, message M
Output: Signature S

1: Select a random integer: $d \in [1, q-1]$
2: Calculate $R(x_1, y_1) = dP$
3: Calculate $R_x = x_1 \ mod \ q$; if $R_x = 0$, go to Step 1
4: Convert the message digest into integer: $h = int(SHA256(M))$
5: Calculate $s = d^{-1}(h + kR_x) \ mod \ q$; if $s = 0$, go to Step 1
6: Define the signature: $S = [R_x, s]$
7: **return** S

Algorithm 3 ECDSA Signature Verification [43]

Curve parameter: Base point P, curve order q
Input: Public key Q, message M, signature S
Output: True/False

1: Convert the message digest into integer: $h = int(SHA256(M))$
2: Calculate $\omega = S[1]^{-1} \ mod \ q$
3: Calculate $u_1 = h\omega \ mod \ q$ and $u_2 = \omega S[0] \ mod \ q$
4: Calculate $U(x_1, y_1) = u_1 P + u_2 Q$
5: **if** $U = 0$ **then**
6: **return** "False"
7: **else**
8: Calculate $v = x_1 \ mod \ q$
9: **if** $S[0] = v$ **then**
10: **return** "True"
11: **else**
12: **return** "False"
13: **end if**
14: **end if**

find k when it is sufficiently large. This difficulty is called elliptic curve discrete logarithm problem (ECDLP) [44], [48].

The ECDSA signature generation and verification mechanisms are presented in Algorithms 2 and 3, respectively [43]. The signing process accepts a private key k and a message M as the input and outputs a two-part signature S. The input to the verification scheme comprises a public key Q, message M, and signature S. If S is verified with Q and M, it means that the signer knows the corresponding private key k.

2.3 Verifiable Random Function

In cryptography, a VRF is a random number generator that generates a pseudo-random output that can be cryptographically verified over a distributed network [31]–[34]. Verifiable randomness is essential to many blockchain applications because its tamper-proof unpredictability provides unbiased outcomes [32]–[39]. Typically, a public–private key pair (i.e., a verification key and secret key) and a seed are passed to a VRF as the input [31]. The VRF then outputs a random number along with a proof. Importantly, creating a proof makes the function verifiable while maintaining the number unpredictability by keeping the private key hidden.

As the name suggests, a VRF satisfies the following properties [31], [32]:

- *Randomness:* The output of a VRF is entirely unpredictable (uniformly distributed) to anyone who does not know the private key. In a VRF, every possible output is equally likely. Randomness is achieved by uniquely combining the private key and seed.
- *Function:* The VRF depends on a mathematical algorithm to produce the random number and the proof, publicly verifying its authenticity.
- *Verifiability:* Only the holder of the VRF private key can compute the random number, whereas anyone with the holder's public key can verify that the VRF output is valid.

3 Proposed Consensus Algorithm

The proposed PoRL algorithm is a modified version of Aura. In Aura, the second round, which involves acceptance sharing, often takes three times longer than the first round. Eliminating this communication round allows PoRL to reduce the overall consensus time to approximately one-third. However, the removal of the acceptance sharing phase raises concerns about network security, as this phase ensures that the network's honest majority validates each block before it is added to the blockchain, preventing double-spending by a step leader.

Although PoA networks assume that all authorities are honest, an authority may act maliciously under external attacks. In Aura, block proposal duties are assigned in a predictable, round-robin manner based on epoch time, allowing authorities to know in advance when they will lead. A malicious authority could exploit this predictability to broadcast a double-spending transaction during its leadership. While it is impossible for other authorities to accept a block without verification in Aura, the absence of acceptance sharing in PoRL necessitates additional security measures.

To address this, PoRL employs a VRF for leader selection, introducing randomness to the process. Unlike the deterministic scheduling in Aura, VRF ensures that step leaders are chosen unpredictably, reducing the risk of premeditated attacks. Even if a malicious leader proposes a double-spending transaction, the randomness in leader selection increases the likelihood that an honest authority will intercept and reject the malicious block during validation. This mechanism, coupled with a voting system to expel double-spenders, ensures that no authority can double-spend a transaction, maintaining the integrity of the network despite the elimination of acceptance sharing.

The PoRL implementation uses the following six threads to run a consortium blockchain network:

- *SendOnlineThread:* This thread sends the joining request when an authority connects to the network.
- *GetOnlineThread:* This thread handles joining requests and keeps track of the authorities' online status.
- *SynchronizeBlockchainThread:* When a crashed or offline authority returns to operation, it can use this thread to synchronize its blockchain with those of the other operational authorities in the network.
- *GetTransactionThread:* This thread receives transactions broadcast by users and stores them in the memory pool (mempool).
- *SendBlockThread:* This thread selects a random leader at each step and broadcasts a block after validating transactions.
- *GetBlockThread:* Non-leader authorities (followers) receive the block proposed by a random leader using

Algorithm 4 Proposed VRF for Random Leader Selection

Input: Authority_list = [A_0, A_1, A_2,...A_{N-1}], genesis block timestamp t_0, current timestamp t, block volume n, step duration δ, previous block hash B_h^-, and shared private keys u, v.
Output: Random leader

Calculate the epoch time: $t_e = t - t_0$
Calculate the regular leader index: $i = \frac{t_e}{\delta}\ mod\ N$
Compute the seed (base) value: $c = int(B_h^-) + i$
Generate a verifiable random number: $r = v - cu$
Determine the random leader index: $I = r\ mod\ N$
return Authority_list[I]

this thread. They add the block to the blockchain after confirming the validity of the block proposer.

The authorties share two private–public key pairs such as $(u, U = uP)$ and $(v, V = vP)$ to generate a verifiable random number r. Users only know the public keys U and V to verify r. The entire process of the PoRL consensus protocol can be broken down into the following sequential steps, which cover everything from receiving transactions to adding validated blocks to the blockchain.

Step 1: Transaction Broadcast

Users broadcast transactions to the blockchain network, which are received and temporarily stored in the *mempool* of each authority node.

Step 2: Monitoring Mempool Size

All authorities individually manage a pending transaction queue $\mathcal{Q}_T$ called the mempool and a blockchain $\mathcal{L}$. Their blockchain state must be synchronous within an epoch time that is the time interval between the creation of the current and genesis (first) blocks. In *SendBlockThread*, the authorities keep track of the mempool size continuously.

Step 3: Random Leader Selection using VRF

When the quantity of transactions in $\mathcal{Q}_T$ is greater than or equal to the block volume (quantity of transactions to store per block) n, they select a random leader (block proposer) using a VRF. Algorithm 4 describes the proposed VRF for random leader selection in PoRL. First, the authorities compute an epoch time t_e as follows [11]–[13]:

$$t_e = t - t_0 \tag{10}$$

where t and t_0 are the timestamps of the current and genesis blocks, respectively.

If δ is the step duration, the regular step leader is identified as A_i, where the leader index i is calculated by the following equation [11]–[13]:

$$i = \frac{t_e}{\delta}\ mod\ N \tag{11}$$

The number of authorities N, the block volume n, and the average computational power of the N authorities all affect how long each step lasts. Therefore, the value of δ is a critical parameter that affects the network performance. It must be carefully chosen to ensure that no authority is hindered by a slow processor or poor internet connection from completing the tasks of transaction validation, block proposal, and acceptance sharing within this time frame. The transaction throughput decreases if δ is either too large or too small. As a result, the network sensitivity must be examined to determine the optimal value of δ.

Algorithm 5 Block Creation [49]

Input: Mempool $\mathcal{Q}_T$, previous block hash B_h^-, previous block timestamp B_t^-, leader's private–public key pair (k_L, Q_L)
Output: Signed block

Define valid transactions: $V_T = [\,]$
for i from 0 to len($\mathcal{Q}_T$) **do**
  if $\mathcal{Q}_T[i].timestamp < B_t^-$ **then**
    if $\mathcal{Q}_T[i]$ is valid **then**
      $V_T.append(\mathcal{Q}_T[i])$
    end if
  end if
end for
Generate a Merkle tree $M_t = \{M_r, M_h\}$ for V_T
for i from 0 to len(V_T) **do**
  Set the transaction index: $T_i = i$
  Set the transaction hash: $T_h = M_h[i]$
  Insert T_i and T_h into $V_T[i]$
end for
Compute the block nonce: $B_n = os.urandom(8).hex()$
Compute the block timestamp: $t = datetime.now()$
Compute the block hash: $B_h = SHA256(B_h^- || M_r || t || B_n)$
Define the block miner: $B_m = Q_L$
Create the block: $B = \{B_i, B_h, B_h^-, B_n, t, M_r, V_T, B_m, B_s\}$
Sign the block: $S_p = sign(k_L, B)$
return $\{B, S_L\}$

The regular leader index is required to set a deadline for the block proposer at each step so that each block is processed within the step duration. However, this index does not identify the PoRL leader; instead, it is one of the parameters of the seed value required to generate a verifiable random number. Second, the previous block hash is converted to an integer, considering the other parameter of the seed value. Third, a seed value c is computed by adding i to the block hash of the last block. Fourth, r is generated by the product of u and c from v. Finally, the random leader index I is determined by performing the modulo N operation with r.

Step 4: Block Proposal by the Selected Leader

The selected random leader validates pending transactions and creates a block B using Algorithm 5 [49]. Pending transactions $\mathcal{Q}_T$, the previous block hash B_h^-, and the leader's private–public key pair (k_L, Q_L) are all accepted as the input. The first step is to filter the transactions in $\mathcal{Q}_T$ that were broadcast before the beginning of the current step and are valid. The comparison of the previous block and transaction timestamps is required to prevent the inclusion of any new transaction in the current block by the leader. Next, a Merkle tree $\{M_r, M_h\}$ is created for the valid transactions V_T, where M_r denotes the Merkle root and M_h stands for a list of Merkle hashes. A transaction index T_i and a transaction hash $T_h \in M_h$ are inserted into the transactions that were broadcast before the beginning of the current step and are valid. These parameters serve as the transaction identifiers. Block B, which contains the block index B_i, block hash B_h, previous block hash B_h^-, nonce B_n, timestamp t, M_r, V_T, block miner $B_m(Q_L)$, and block size B_s, is formed once all transactions have been indexed. If no transaction satisfies the validation criteria, an empty

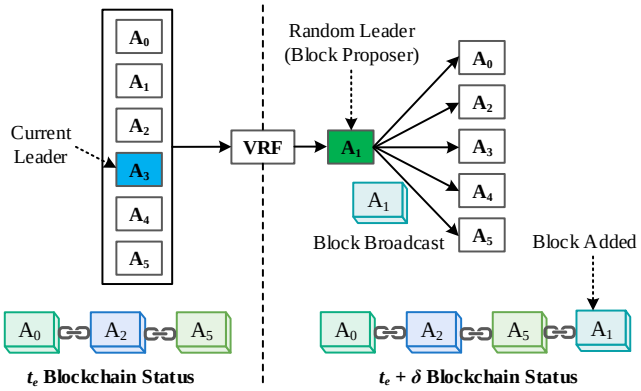

Fig. 2. Consensus process in proposed PoRL algorithm.

block is created. For the followers to authenticate the block, the leader must sign the block with its private key k_L and generate a signature S_L before broadcasting.

After the block is created, the leader broadcasts the block (along with its signature $\{B, S_L\}$) to all other authority nodes. If there are insufficient transactions in the mempool, the leader broadcasts an empty block to indicate that it is operational but lacks enough transactions for a full block.

Step 5: Block Validation and Storage by Followers

Each follower node receives the block and performs the following validation checks: 1) Verify the block signature S_L with B_m to ensure that it was signed by the correct leader. 2) Ensure that the block proposer is the selected random leader. 3) Check that all transactions included in the block are valid.

If the block passes all validation checks, it is added to the local copy of the blockchain $\mathcal{L}$. Transactions that were included in the block are removed from the mempool.

Step 6: Invalid Block Handling

If the proposed block is invalid, the followers do not add it to their local blockchains. Instead, they cast a vote against the block proposer. If the majority of the followers agree that the proposer is malicious or faulty, the proposer is kicked out of the network. This voting mechanism ensures that faulty nodes are removed through a consensus process, maintaining the integrity of the blockchain.

Step 7: New Leader Selection

The leader changes when the step duration is over, a new random leader is selected using the same VRF described earlier, and the process repeats for the next block proposal.

Fig. 2 illustrates the overall consensus process in PoRL. The authority A_3 is selected as the regular leader using the round-robin method. However, PoRL does not directly utilize this regular leader. Instead, the VRF selects a random leader, A_1, to propose the block. This process ensures that no authority can predict when they will become the leader, enhancing security. Once selected, A_1 proposes a block and broadcasts it to the network. Following this, the block is validated and subsequently added to the blockchain.

Verification and Randomness

Anyone in the network can verify the correctness of the random number by knowing the shared public keys of the authorities and checking whether the following equation is satisfied:

$$V = rP + cU \tag{12}$$

where c is calculated using the block timestamp B_t and transactions in the block.

However, no one can compute a correct random number without knowing the shared private keys of the authorities. As a result, no adversary can make a false transaction by targeting an authority because they cannot predict the step leader. As soon as a new block is added to the blockchain, the previous hash of the last block is updated, which changes r at each step and contributes to randomness.

Improvement

In Aura, followers share validation results for every block, whether valid or invalid. This can be inefficient, as it requires verifying multiple signatures for every block. PoRL improves this process by only sharing validation results when a block is deemed invalid. This reduces communication overhead while maintaining security through random leader selection.

Compared to Clique, which lacks a random leader and allows multiple proposers to submit blocks per round, PoRL enhances both efficiency and security through its randomized leader selection approach. PoRL's single-leader mechanism reduces the risk of forks and lowers the number of messages per step, accelerating the consensus process.

Additionally, PoRL resists time and order manipulation attacks by selecting leaders randomly rather than following a round-robin schedule. The unpredictability in leader selection strengthens the system's resilience against malicious actors by preventing the foresight of the next leader.

4 Fault Tolerance Analysis

The proposed algorithm operates without an explicit acceptance round unlike Aura, meaning that nodes individually verify blocks and vote to remove faulty nodes when they detect malicious behavior. However, the fault tolerance of PoRL remains the same as traditional PoA.

Fault Tolerance Threshold

Each follower independently validates the block proposed by the selected leader. If the block is valid, it is added to their local copies of the blockchain. However, if the block is invalid, a voting mechanism is triggered, and the followers vote to remove the malicious leader. If the network contains N nodes, at least $N/2$ votes are required to remove the faulty leader from the network. Assuming that the faulty followers would not vote against the faulty leader but rather certify the malicious block, the fault tolerance threshold is:

$$f < \frac{N}{2}$$

where f is the number of faulty nodes that can be present in the network.

This ensures that the algorithm can tolerate Byzantine faults (arbitrary or malicious behavior) as long as long as more than half of the nodes are honest.

Leader Selection Probability

Leader selection is performed using a VRF, ensuring that the leader is chosen randomly. The probability that a faulty node is selected as the leader in a given round is:

$$P_{\text{faulty leader}} = \frac{f}{N}$$

Similarly, the probability that an honest node is selected as the leader is:

$$P_{\text{honest leader}} = \frac{N-f}{N}$$

Given that $N > 2f$, the likelihood of selecting an honest leader is always greater than that of selecting a faulty one.

Probability of Multiple Consecutive Faulty Leaders

The probability that multiple consecutive rounds select faulty leaders is exponentially small. For example, the probability that two consecutive faulty leaders are selected is:

$$P_{\text{2 faulty leaders}} = \left(\frac{f}{N}\right)^2$$

For k consecutive faulty leaders, the probability is:

$$P_{\text{k faulty leaders}} = \left(\frac{f}{N}\right)^k$$

Since $f < \frac{N}{2}$, the likelihood of multiple consecutive faulty leaders is extremely low, making it unlikely that faulty nodes will disrupt the system for an extended period.

Liveness Condition

The system ensures liveness by allowing honest nodes to continue validating and proposing blocks even in the presence of faulty nodes. The probability of selecting an honest leader in any given round is:

$$P_{\text{honest leader}} = \frac{N-f}{N}$$

As long as the majority of nodes are honest, the system will remain live, meaning that it will continue to process transactions and add valid blocks to the blockchain.

5 Security Model

A security model for the proposed algorithm can be defined using a game with a PoRL oracle $\mathcal{O}^{PoRL}$, Type I adversary, and Type II adversary. These adversaries are distinguished by unique capabilities:

- *Type I adversary:* This entity is a malicious user who cannot access the private keys of any authority but knows the authorities' public keys.
- *Type II adversary:* This entity is a malicious random step leader who knows the authorities' shared private keys u and v required to generate a verifiable random number.

To satisfy the security notions for the PoRL algorithm, the following games are played:

Game I: This game is played between a Type I adversary $\mathcal{A}_{\mathcal{I}}$ and a challenger $\mathcal{C}$. In this game, $\mathcal{A}_{\mathcal{I}}$ tries to compute a random number r that satisfies the verifying equation defined for PoRL. During the interaction between $\mathcal{A}_{\mathcal{I}}$ and $\mathcal{C}$, $\mathcal{C}$ notes all queries with answers in a list.

Initialization: $\mathcal{C}$ defines the curve parameters $\{E, p, q, P\}$.

Query: $\mathcal{A}_{\mathcal{I}}$ sends a seed c and the shared public keys of the PoRL authorities U and V to $\mathcal{C}$. $\mathcal{C}$ searches for the private keys k_1 and k_2 so that $U = k_1 P$ and $V = k_2 P$ hold. Upon finding out both keys, $\mathcal{C}$ computes the random number as $r = k_2 - ck_1$ and returns it to $\mathcal{A}_{\mathcal{I}}$.

Result: $\mathcal{A}_{\mathcal{I}}$ wins Game I if they can determine the corresponding private keys of U and V by solving the ECDLP so that $V = rP + cU$ holds.

Game II: In this game, $\mathcal{A}_{\mathcal{I}}$ tries to falsify the randomness property of the random number r in PoRL.

Initialization: $\mathcal{C}$ downloads the blockchain and defines the curve parameters $\{E, p, q, P\}$.

Query: $\mathcal{A}_{\mathcal{I}}$ sends two arbitrary private keys $k_1, k_2 \in \mathbb{F}_q$ to $\mathcal{C}$. $\mathcal{C}$ collects all the previous hashes $\forall B_h^-$ of the blocks in $\mathcal{L}$ and possible leader indices $\forall i$. Then, the challenger computes each corresponding seed $c_i = SHA256(B_h^-) + i$ and random number $r_i = k_1 - c_i k_2$, where $i \in \mathbb{Z}^*$, and stores each random number in a list L_r. Then, it searches L_r for two random numbers r_i and r_j that are equal, such that $r_i = r_j; i \neq j$, and returns the search results to $\mathcal{A}_{\mathcal{I}}$.

Result: $\mathcal{A}_{\mathcal{I}}$ wins Game II if the SHA256 hash function is not collision-resistant, i.e., $SHA256(x) = SHA256(y)$ for $x \neq y$.

Game III: In this game, $\mathcal{A}_{\mathcal{I}}$ challenges the verifiability property of the random number r in PoRL.

Initialization: $\mathcal{C}$ defines the curve parameters $\{E, p, q, P\}$.

Query: $\mathcal{A}_{\mathcal{I}}$ sends the corresponding random number r_i and seed c_i of a block B_i stored in the blockchain to $\mathcal{C}$, where $i \in \mathbb{Z}^*$. They also delivers the authorities' shared public keys U and V to $\mathcal{C}$. $\mathcal{C}$ checks whether the equation $V = r_i P + c_i U$ holds. It returns the verification result to $\mathcal{A}_{\mathcal{I}}$.

Result: $\mathcal{A}_{\mathcal{I}}$ wins Game III if r_i is not computed using the authorities' shared private keys u and v, i.e., $r_i \neq v - c_i u$ or $v \neq r_i + c_i u$.

Game IV: This game is played between a Type II adversary $\mathcal{A}_{\mathcal{II}}$ and a challenger $\mathcal{C}$. In this game, $\mathcal{A}_{\mathcal{II}}$ attempts to double-spend a transaction during their leadership.

Initialization: $\mathcal{C}$ downloads the blockchain and defines the curve parameters $\{E, p, q, P\}$, and step duration δ.

Query: $\mathcal{A}_{\mathcal{II}}$ broadcasts a double-spending transaction T^* to the network during their leadership and sends the private keys u and v to $\mathcal{C}$. $\mathcal{C}$ creates a block B^* containing T^* and returns the block to $\mathcal{A}_{\mathcal{II}}$. Finally, $\mathcal{A}_{\mathcal{II}}$ broadcasts B^* to the network.

Result: $\mathcal{A}_{\mathcal{II}}$ wins Game IV if the arrival of T^* takes place earlier than the arrival of the previous block to all the nodes, which is impossible according to the leader selection method.

Definition 2: In $\mathcal{O}^{PoRL}$, if no adversary has the non-negligible advantage to solve the ECDLP, then the PoRL algorithm is secure as long as the network is fault-tolerant.

6 Security Analyses

Theorem 1. *No unauthorized party can compute the random number r in the PoRL algorithm with the notion that the ECDLP is unbreakable.*

Proof: From Game I in $\mathcal{O}^{PoRL}$, it is clear that the only way for a Type I adversary $\mathcal{A}_{\mathcal{I}}$ to compute the random number r is to solve the ECDLP. The simplest algorithm to solve the ECDLP is the exhaustive search in which $\mathcal{A}_{\mathcal{I}}$ computes the sequence of points $P, 2P, 3P, 4P, ...$ until Q is encountered. It requires approximately q point additions in the worst case and $q/2$ point additions on average. The best general-purpose algorithm known for solving the ECDLP is Pollard's rho algorithm [40], which has an expected running time of $\sqrt{\pi q}/2$ point additions. The algorithm's speed can further be boosted by employing m processors in parallel, where the speedup is linear in the number of processors employed. For the parallelized version of Pollard's rho, the computation time is reduced to $\sqrt{\pi q}/(2m)$ point additions.

It is noteworthy that there is no mathematical proof that the ECDLP is intractable. Theoretically, it is possible to solve the ECDLP, but practically, it is difficult over a large prime field [45]. To combat this attack, the elliptic curve parameters should be carefully chosen with a sufficiently large curve order (e.g., $q \geq 2^{160}$) so that the attacker requires an infeasible amount of computation. On our computer (CPU: Intel Core i5-10400 @ 2.9 GHz, RAM: 64 GB), a point addition takes 8 μs. In this viewpoint, if we employ 10000 computers of such specifications together on the internet to solve the ECDLP over 256-bit $\mathbb{F}_q$, it will require approximately 3×10^{34} point additions $\approx 7.65\times10^{21}$ years, which is clearly infeasible.

Theorem 2. *Proof of randomness: As the blockchain $\mathcal{L}$ updates for increasing blocks, the VRF generates a unique random number at each step that makes the block proposer unpredictable.*

Proof: Users continuously broadcast transactions to the blockchain network, which are stored in increasing blocks. When a new block is added to $\mathcal{L}$, the seed value c and the random number r change for different hash values of the previous blocks. It makes the block proposer unpredictable.

Theorem 3. *Proof of verifiability: Any participant can verify the validity of a block miner B_m by knowing the corresponding random number r and authorities' shared public keys V and U.*

Proof: Recall (6) where a participant verifies the validity of the random number r by the following equation:

$$V = rP + cU \tag{13}$$

According to elliptic curve cryptography, (7) can be written as follows:

$$vP = rP + c(uP) \tag{14}$$

$$v = r + cu \tag{15}$$

$$r = v - cu \tag{16}$$

(9) proves that r was not generated by any adversary; rather, it was computed by an authority in the network who knows the private keys v and u. Without the knowledge of v and u, a Type I adversary $\mathcal{A}_{\mathcal{I}}$ cannot compute r.

To verify the validity of the block miner B_m, the verifier generates the VRF output using r and checks whether the output is B_m or not.

Theorem 4. *No authority can double-spend a transaction after being randomly selected as the step leader.*

Proof: At each step, one of the N authorities is randomly selected as the block proposer. To mitigate the risk of a selected leader including double-spending transactions in their proposed block, a mechanism has been implemented, as shown in line 3 of Algorithm 5, that compares the transaction timestamps with the previous blocks timestamp. Transactions with timestamps greater than that of the previous block are deferred from inclusion in the current block. They are held until their timestamps align with the previous block's timestamp in subsequent rounds. Consequently, the leader cannot include any new transaction (either legitimate or double-spent) that was not present in the mempool before their leadership. As a result, the current version of our algorithm is double-spending proof.

Furthermore, all authorities must be honest, and they are accountable for their malfunctions in the permissioned blockchain network.

Theorem 5. *A distributed denial of service (DDoS) attack may lengthen block times and reduce transaction throughput, but it cannot terminate the PoRL network.*

Proof: A DDoS attack is a common threat in decentralized networks [50], [51]. PoA is only susceptible to such attacks, similar to BFT and crash fault tolerance consensus algorithms, if the IP addresses of the validators are somehow leaked outside the private network and no access control measures are implemented. As block delivery in PoA relies on a stable leader at each step, DDoS attacks targeting step leaders repeatedly may lengthen block times and reduce transaction throughput. Poor network connectivity for certain nodes can exacerbate these effects. Nonetheless, DDoS attacks can only delay the block generation process; they cannot force consensus nodes to accept malicious transactions or invalid blocks.

To mitigate the risk of DDoS attacks, access control mechanisms [52], [53] can be employed to deny transactions from unauthorized users. Additionally, network monitoring systems can detect and block traffic from sources that exhibit repetitive or high-volume packet sending within short time intervals, which may indicate malicious behavior [54]. Deploying redundant (replica) nodes as backups for critical validators further enhances the networks resilience by ensuring continuity of service even if some nodes are temporarily disabled by an attack. These replica nodes can dynamically take over the responsibilities of compromised or overwhelmed validators, reducing the likelihood of network paralysis. Together, these strategies safeguard the stability and functionality of PoA networks against potential DDoS attempts.

7 Performance Analyses

7.1 Implementation

The proposed algorithm was implemented using Python with six consensus nodes called authorities A_0, A_1, A_2, A_3, A_4, and A_5. The nodes were interconnected through

TABLE 1
Machine Specifications and Computational Power of the Six Authorities

Parameters	Authority A_0	Authority A_1	Authority A_2	Authority A_3	Authority A_4	Authority A_5
CPU	Intel Core i5-10400	AMD Ryzen 7 4700U	Intel Core i7-11700	AMD Ryzen 7 1700	Intel Core i7-6700	Intel Core i7-6500U
Processor speed (GHz)	2.90	2.00	2.50	3.29	3.40	2.50
RAM (GB)	64	16	64	16	16	16
Number of cores	6	8	8	8	4	2
Block preparation time (s)	0.71	0.89	0.53	0.82	0.82	1.40
Block validation time (s)	1.35	1.37	1.03	1.63	1.42	1.78
Data rate (Kbps)	377.6	246.8	251.7	205.3	133.2	190.1

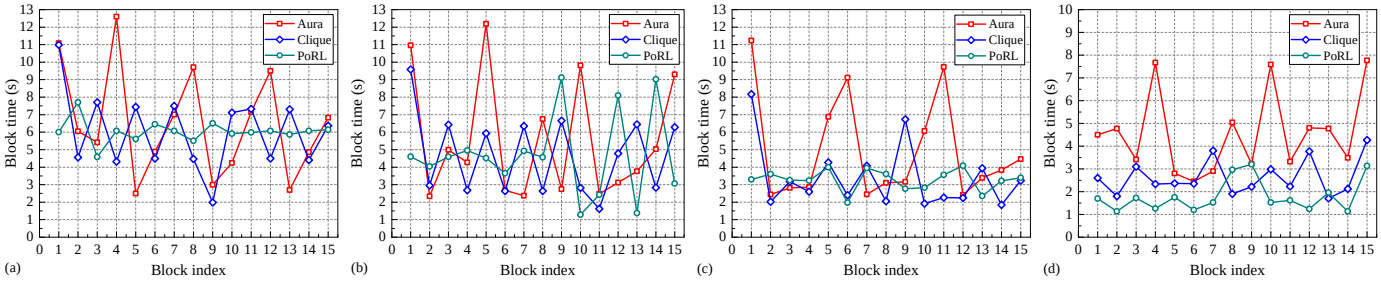

Fig. 3. Block times of the first 16 blocks in Aura, Clique, and PoRL; Tx rate: (a) 17 TPS, (b) 24 TPS, (c) ≈35 TPS, and (d) 86 TPS.

a local area network. The network traffic monitoring software Wireshark was used to measure the data rate. For comparison, the experimental data of Aura and Clique are adapted from our previous research study [12], where the experiments were conducted in the same environment. The authorities' computational power and machine specifications are presented in Table 1. The nodes can be arranged in the following way depending on their processing speed:

$$A_2 > A_0 > A_4 > A_1 > A_3 > A_5 \tag{17}$$

The average latency required to compute the block hash and validate each transaction included in a block is referred to as the block preparation time. The average latency needed to verify the accuracy of a block hash and the validity of each transaction contained in a block is called the block validation time.

A transaction on the majority of blockchains contains the sender's digital signature, which must be verified with the sender's public key using ECDSA [43]. In the transaction validation process, the execution of ECDSA takes the longest time. The time required to validate a transaction is nearly identical to the time needed to validate a digital signature. Therefore, when conducting the experiments, we treated digital signatures as transactions. To generate a transaction, a random private–public key pair (Sk, Pk) was first created. Then, a signature S was produced by signing a random message M with Sk. The transaction T was created by listing Pk, M, S, and the current timestamp t such that $T = [Pk, M, S, t]$. Using a loop operation, sample transactions were generated continuously. Generated transactions were broadcast to the blockchain network using the user datagram protocol. With a real-time implementation in mind, the Python time-sleep function was used to vary the transaction arrival rate. Broadcast transactions were stored in the authorities' mempools before adding to the blockchain.

The block volume denoted as n was fixed to 100 transactions per block. A random leader began to prepare a block as soon as the network had 100 transactions pending. By verifying the sender's signatures using the sender's public keys and messages, the leader first confirmed the pending transactions. The leader then checked whether the public keys were already used in any transactions recorded on the blockchain. It was necessary to avoid double-spending a transaction. When a block was created with confirmed transactions, it was distributed to the other authorities in the network for storage.

7.2 Results and Performance Comparison

To evaluate the network performance, the following parameters were taken into account:

- *Block time:* This is the time interval between two consecutive blocks being stored in the blockchain.
- *Epoch time:* Epoch time is determined by subtracting the genesis (first) block timestamp from the current block timestamp.
- *Transaction arrival rate*: The number of transactions arriving at the network per second is defined by this rate.
- *Transaction throughput:* This is the number of transactions that the network can process per second.
- *Utilization:* This is defined as the ratio of transaction throughput to the transaction arrival rate over the network. It figures out how effectively the network processes broadcast transactions [55].

Fig. 3 shows the block times of blocks 1 to 15 (for simple illustration) in Aura, Clique, and PoRL. Block times fluctuated because of the authorities' performance variation. The block proposers delivered empty blocks, which were not stored in the blockchain when the transaction arrival rate was 17 TPS. They waited for the mempool to fill

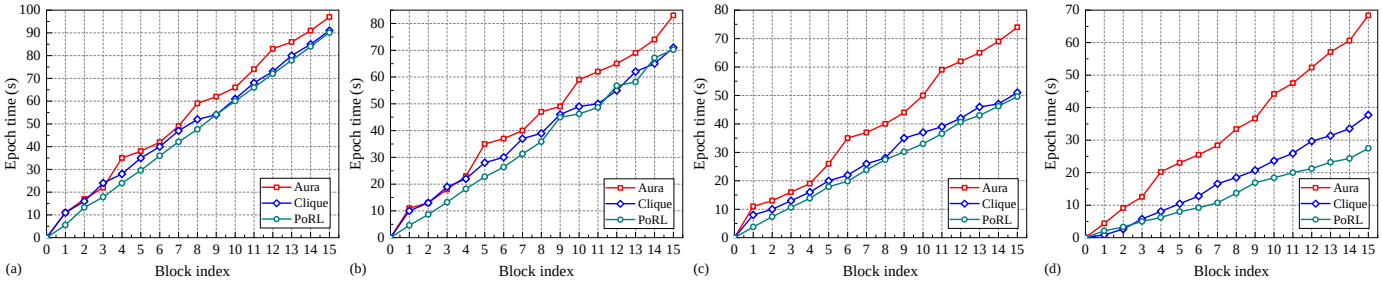

Fig. 4. Epoch times of the first 16 blocks in Aura, Clique, and PoRL; Tx rate: (a) 17 TPS, (b) 24 TPS, (c) ≈35 TPS, and (d) 86 TPS

up with 100 transactions. Thus, block times grew during the network's long pauses between block arrivals. With an increased transaction arrival rate, there were fewer empty blocks and shorter wait times for the authorities.

Fig. 4 compares the algorithms' speed in terms of epoch time. The Aura network took 97 s to store 16 blocks in the blockchain at 17 TPS transaction arrival rate, whereas Clique and PoRL required 91 and 90 s, respectively, to process the same number of blocks. The same number of blocks containing the same number of transactions were processed in these networks in 74, 51, and 28 s, respectively, when the transaction arrival rate was 86 TPS. The transaction throughput was calculated by the division of the number of transactions stored in the blockchain and the epoch time. With a higher transaction arrival rate, the block epoch times shortened and the speed disparity between these algorithms widened. As more transactions were broadcast per second, lower epoch times resulted from the authorities processing blocks more quickly and not waiting for transactions. Even if a speedier leader added a block to the blockchain quickly at an Aura or PoRL step, the following leader could not propose a block before the end of the step. The Clique network, however, allowed a block to begin processing as soon as its previous block had been stored in the blockchain. Clique is hence quicker than Aura, as was seen in all of these instances. In PoRL, block receivers could directly accept the block proposed by a random leader. Thus, this algorithm provided higher speed than Aura and Clique, especially in high workload conditions.

Fig. 5 illustrates the impact of step duration on epoch time. To ascertain the optimal step duration for PoRL, we conducted a sensitivity analysis by varying this parameter. Notably, with a step duration of 1.5 seconds, the algorithm demonstrated the shortest time required to mine 20 blocks. Conversely, when the step duration was set to 1 or 2 seconds, the epoch time for the same block index notably increased. Hence, considering a block volume of 100 transactions, the optimal step duration for PoRL stands at 1.5 seconds. It is crucial to acknowledge that these findings are specific to our experimental setup and may vary depending on network computing power, such as the specifications of individual machines.

The overall performance comparison of the algorithms is presented in Table 2. PoRL provides higher transaction throughput and utilization than Aura and Clique. Due to greater disparities between transaction arrival rate and throughput, network utilization declines as transaction arrival rate increases. For the six authorities, PoRL completes each step at a cost of only 5 messages, compared to Aura and Clique who handle 30 and 10 messages each step, respectively. However, the computational complexity and level of fault tolerance of all these algorithms are the same. Since only the leader can propose a block at each step in Aura and PoRL, a turn is lost if the leader is offline. In contrast, at every Clique step, an assistant block proposer exists. If no block comes from the leader, the assistant's block is accepted. As a result, Clique has greater availability. Because each block needs a minimum of three signatures from the six authorities before it can be added to the blockchain, Aura offers higher security than Clique and PoRL. Each authority receives the same block multiple times during the Aura acceptance-sharing phase, minimizing the possibility of missing a block but requiring greater bandwidth and latency. In Clique, a *fork* is a frequent issue that makes it impossible for the authorities to synchronize their ledgers. The consistency of the blockchain data is impacted by this issue. On the contrary, no *fork* occurs in Aura and PoRL. Therefore, these algorithms provide greater data consistency than Clique.

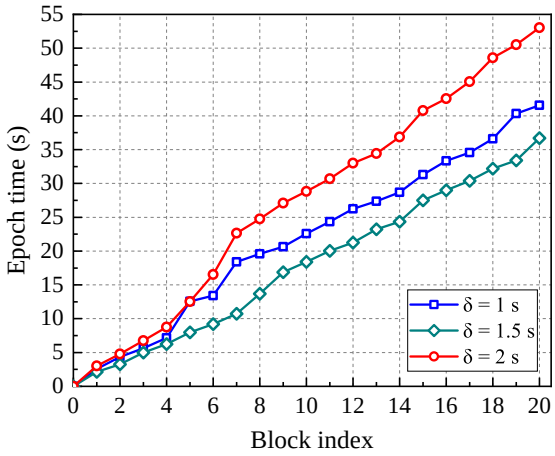

Fig. 5. Variation in the epoch time for the variable step duration in PoRL.

8 Limitation and Scope

A limitation of the proposed PoRL algorithm is that the security of the network relies on the randomness provided by the adopted VRF. The level of randomness improves with an increase in the number of authorities or the size

TABLE 2
Performance Comparison of The Aura, Clique, And PoRL Implementations for Six Consensus Nodes

Implementations	Aura				Clique				PoRL			
Transaction arrival rate (TPS)	17	24	33	86	17	24	37	86	17	23	33	86
Average block time (s)	6.5	5.5	4.9	4.5	6	4.7	3.4	2.6	6	4.6	3.3	1.7
Transaction throughput (TPS)	15	18	20	21	16	21	29	39	16	21	30	54
Utilization	88%	75%	61%	24%	94%	88%	78%	45%	94%	91%	90%	62%
Computational complexity	$\mathcal{O}(n)$				$\mathcal{O}(n)$				$\mathcal{O}(n)$			
No. of messages per step	30				10				5			
Availability	Low (single proposer)				High (multiple proposers)				Low (single proposer)			
Consistency	High (no fork)				Low (fork)				High (no fork)			
Security	High (multiple proofs)				Low (single proof)				Moderate (single proof with randomness)			
Transaction size (B)	360				360				360			
Block size (kB)	35.5				35.5				35.5			

of the network. Therefore, PoRL is not ideal for networks with a limited number of consensus nodes. However, it is more suited for large-scale networks with high transaction volumes that experience low utilization issues.

If the transaction throughput is lower than the transaction arrival rate, the mempool size increases rapidly. As a result, users have to wait a long time for their transactions to be confirmed by the validators [55]. Thus, a network with insufficient utilization may fail to meet user demands. From this point of view, PoRL outperforms Aura and Clique.

We had a limited number of computers available to conduct the experiment. Thus, we could not perform experiments with a variable number of authorities. However, we argue that the transaction throughput in Aura and Clique will drastically decrease with the number of authorities. If the number of authorities increases, the number of acceptances required to confirm a block in Aura and the number of block proposers at each step and occurrence of *forks* in Clique increase. By contrast, the transaction throughput will not remarkably decrease with the number of authorities in PoRL because authorities do not wait for other's confirmation and there is no *fork* to solve in the network.

9 Conclusion

This paper proposed a novel Proof of Authority (PoA)-based consensus algorithm named Proof of Random Leader (PoRL), which leverages a Verifiable Random Function (VRF) to introduce randomness in leader selection. The proposed PoRL algorithm addresses the limitations of existing PoA variants, such as Aura and Clique, by mitigating issues related to low throughput and predictable leader selection. Using a VRF, PoRL ensures unpredictability in the election of block proposers, thereby enhancing the security and robustness of the consensus process in permissioned blockchain environments. A comparative analysis of PoRL with Aura and Clique was performed, providing valuable insights for the adoption of these algorithms in permissioned blockchain systems. The PoRL algorithm was implemented in Python and tested using six consensus nodes with varying computational capacities within a local area network. The experimental evaluation considered multiple performance parameters, including availability, security, and transaction throughput. The results indicated that Clique exhibits the highest availability, Aura provides the strongest security guarantees, and PoRL offers the fastest transaction throughput and the lowest consensus time. It is important to recognize that no consensus algorithm is universally optimal; each has distinct advantages and limitations depending on the specific requirements of the blockchain system. This study offers theoretical and empirical contributions that are valuable for practitioners and researchers working with PoA-based permissioned blockchains. For future research, we plan to expand the experimental environment to include a variable number of consensus nodes across a distributed network infrastructure. This will enable a deeper understanding of PoRLs performance and scalability under diverse network conditions, contributing to the optimization of PoA protocols for real-world blockchain deployments.

Acknowledgment

This work was partially supported by the Seoul R&BD Program (VC230019) through the Seoul Business Agency (SBA) funded by the Seoul Metropolitan Government; Korea University research grant; Korea University Computer Science Brain Korea 21 (BK21) Four funding grant; Institute of Information & communications Technology Planning & Evaluation (IITP), Korean government (MSIT), Grant No. 2021-0-00177; National Research Foundation of Korea (NRF), Grant No. NRF-2021R1A2C2012476.

Deceleration of Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] B. Lashkari and P. Musilek, "A comprehensive review of blockchain consensus mechanisms," *IEEE Access*, vol. 9, pp. 43620–43652, Mar. 2021.
[2] S. Kaur, S. Chaturvedi, A. Sharma, and J. Kar, A research survey on applications of consensus protocols in blockchain, *Secur. Commun. Netw.*, vol. 2021 art. no. 66937312021, Jan. 2021.
[3] Y. Xiao, N. Zhang, W. Lou, and Y. T. Hou, A survey of distributed consensus protocols for blockchain networks, *IEEE Commun. Surv. Tutor.*, vol. 22, no. 2, pp. 1432–1465, Jan. 2020.

[4] M. M. Merlec and H. P. In. "Blockchain-based decentralized storage systems for sustainable data self-sovereignty: A comparative study," *Sustainability*, vol.16, no.17, p. 7671, Sep. 2024.

[5] S. Bouraga, "A taxonomy of blockchain consensus protocols: A survey and classification framework," *Expert Syst. Appl.,* vol. 2021, no. 168, p. 114384, Apr. 2021.

[6] C. Cachin and M. Vukoli, "Blockchain consensus protocols in the wild," *arXiv preprint,* July 2017. [Online]. Available: https://arxiv.org/pdf/1707.01873.pdf.

[7] B. Cao, Z. Zhang, D. Feng, S. Zhang, L. Zhang, M. Peng, and Y. Li, Performance analysis and comparison of PoW, PoS and DAG based blockchains, *Digit. Com. Netw.,* vol. 6, no. 4, pp. 480–485, Nov. 2020.

[8] G. A. F. Rebello, G. F. Camilo, L. C. B. Guimares, L. A. C. Souza, G. A. Thomaz, and O. C. M. B. Duarte, A security and performance analysis of proof-based consensus protocols, *Ann. Telecommun.,* pp. 1–21, Nov. 2021, doi: 10.1007/s12243-021-00896-2.

[9] N. Tomi, "A review of consensus protocols in permissioned blockchains," *J. Comp. Sci. Res.,* vol. 2, no. 3, pp. 19–26, Apr. 2021.

[10] M. M. Merlec, M. M. Islam, Y. K. Lee, and H. P. IN, A consortium blockchain-based secure and trusted electronic portfolio management scheme, *Sensors,* vol. 2022, no. 22, p. 1271, Feb. 2022.

[11] I. Barinov, V. Baranov, and P. Khahulin (2018), *PoA network white paper.* Accessed: 16 Dec. 2024. [Online]. Available: https://github.com/poanetwork/wiki/wiki/POA-Network-Whitepaper.

[12] M. M. Islam, M. M. Merlec, and H. P. In, "A comparative analysis of proof-of-authority consensus algorithms: Aura vs Clique," in *Proc. IEEE Int. Conf. Serv. Comput. (SCC),* Barcelona, Spain, July 2022, pp. 327–332.

[13] S. D. Angelis, L. Aniello, R. Baldoni, F. Lombardi, A. Margheri, and V. Sassone, "PBFT vs proof-of-authority: Applying the CAP theorem to permissioned blockchain, in *Proc. Italian Conf. Cyber Secur.,* Milan, Italy, June 2018, pp. 111.

[14] J. Sousa, A. Bessani, and M. Vukolic, "A Byzantine fault-tolerant ordering service for the hyperledger fabric blockchain platform," in *Proc. 48th Annu. IEEE Int. Conf. Dependable Syst. Netw. (DSN),* June 2018, pp. 51–58.

[15] M. Castro and B. Liskov, Practical byzantine fault tolerance and proactive recovery, *ACM Trans. Comput. Syst.,* vol. 20, no. 4, pp. 398461, Nov. 2002.

[16] M. Henrique. "The Istanbul BFT consensus algorithm," *arXiv preprint,* May 2020. [Online]. Available: https://arxiv.org/pdf/2002.03613.pdf.

[17] S. D. Angelis, "Assessing security and performances of consensus algorithms for permissioned blockchains," *arXiv preprint,* May 2018. [Online]. Available: https://arxiv.org/pdf/1805.03490.pdf.

[18] G. A. F. Rebello, et al., Security and performance analysis of Quorum-based blockchain consensus protocols, 2020. Accessed on: 9 May 2022. [Online]. Available: https://www.gta.ufrj.br/ftp/gta/TechReports/RCG20c.pdf.

[19] S. Sondhi, S. Saad, K. Shi, M. Mamun, and I. Traore, "Chaos engineering for understanding consensus algorithms performance in permissioned blockchains," in *Proc. IEEE Intl Conf on Dependable, Autonomic and Secure Computing,* Canada, 2021, pp. 51–59.

[20] M. Mazzoni, A. Corradi, and V. D. Nicola, "Performance evaluation of permissioned blockchains for financial applications: The ConsenSys Quorum case study," *Blockchain Res. Appl.,* vol. 1, no. 4, p. 100026, May 2021.

[21] I. Homoliak, S. Venugopalan, D. Reijsbergen, Q. Hum, R. Schumi, and P. Szalachowski, The security reference architecture for blockchains: Toward a standardized model for studying vulnerabilities, threats, and defenses, *IEEE Commun. Surv. Tutor.,* vol. 23, no. 1, pp. 341390, Oct. 2021.

[22] X. Wu, J. Chang, H. Ling, and X. Feng, "Scaling proof-of-authority protocol to improve performance and security," *Peer-to-Peer Netw. Appl.*, vol. 15, pp. 2633–2649, 2022.

[23] D. Hanggoro, J. H. Windiatmaja, A. Muis, R. F. Sari, E. Pournaras, "Energy-aware Proof-of-Authority: Blockchain Consensus for Clustered Wireless Sensor Network." *Blockchain: Research and Applications*, p. 100211, Jun 2024.

[24] P. Szilgyi, "EIP-225: Clique proof-of-authority consensus protocol, Ethereum improvement proposals," no. 225, Mar. 2017. [Online]. Available: https://eips.ethereum.org/EIPS/eip-225.

[25] S. Alrubei, E. Ball and J. Rigelsford, "HDPOA: Honesty-Based Distributed Proof of Authority Via Scalable Work Consensus Protocol for IoT-Blockchain Applications, *Computer Networks*, Vol. 217, p. 109337, Nov. 2022.

[26] Wu, X., Chang, J., Wang, Z. et al. "DyPoA: enhanced PoA protocol with a dynamic set of validators for IoT," *Cluster Computing*, vol. 27, pp. 1252712545, June 2024.

[27] X. Zhang, R. Li, Q. Wang, Q. Wang, and S. Duan, "Time-manipulation Attack: Breaking Fairness against Proof of Authority Aura," *Proc. ACM Web Conf. (WWW '23)*, April 2023, pp. 2076–2086.

[28] Q. Wang, R. Li, Q. Wang, S. Chen, and Y. Xiang, "Exploring Unfairness on Proof of Authority: Order Manipulation Attacks and Remedies," in *Proc. 2022 ACM on Asia Conf. Comput. Commun. Secur. (ASIA CCS '22)*, Association for Computing Machinery, New York, NY, USA, 2022, pp. 123–137.

[29] X. Zhang, Q. Wang, R. Li, and Q. Wang, "Frontrunning Block Attack in PoA Clique: A Case Study," *Proc. IEEE Int. Conf. Blockchain & Cryptocurrency (ICBC)*, Shanghai, China, 2022, pp. 1–3.

[30] P. Ekparinya, V. Gramoli, and G. Jourjon, "The attack of the clones against proof-of-authority," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, San Diego, CA, USA, Feb. 2020, pp. 1–14.

[31] Chainlink (June 2022). *Verifiable Random Function (VRF),* Accessed: 10 Oct. 2024. [Online]. Available: https://blog.chain.link/verifiable-random-function-vrf.

[32] L. Cao, P. Wang, Y. Hu, and Z. Sun, "Consensus Algorithms Based on Collusion Resistant Publicly Verifiable Random Number Seeds," *SSRN,* Accessed: Mar. 11, 2024. [Online]. Available: http://dx.doi.org/10.2139/ssrn.4333427.

[33] G. Guo, Zhu, Y., Chen, E., Zhu, G., Ma, D., and Chu, W.C, "Continuous improvement of script-driven verifiable random functions for reducing computing power in blockchain consensus protocols,"*Peer-to-Peer Netw. Appl.*, vol. 15, pp. 304–323, 2022.

[34] R. Bezuidenhout, W. Nel, and J. M. Maritz, "Embedding tamper-resistant, publicly verifiable random number seeds in permissionless blockchain systems," *IEEE Access*, vol. 10, pp. 39912–39925, 2022.

[35] W. Werapun, T. Karode, J. Suaboot, T. Arpornthip, and E. Sangiamkul, "NativeVRF: A Simplified Decentralized Random Number Generator on EVM Blockchains," *Systems*, vol. 11, no. 7, p. 326, 2023.

[36] Wang, Ping, Weiqian Chen, and Zhiwei Sun, "Consensus algorithm based on verifiable randomness," *Inf. Sci.,* vol. 608, pp. 844-857, 2022.

[37] H. Wang and W. Tan,"Block proposer election method based on verifiable random function in consensus mechanism," in *Proc. IEEE Int. Conf. Prog. Inf. Comput. (PIC),* Shanghai, China, Feb. 2021, pp. 304–308.

[38] Y. Li *et al.,* "An extensible consensus algorithm based on PBFT," in *Proc Int. Conf. Cyber. Distrib. Comput. Knowl. Discov. (CyberC),* Guilin, China, Jan. 2020, pp. 17–23.

[39] Shuang Yao and Dawei Zhang, "An anonymous verifiable random function with applications in blockchain," *Wirel. Commun. Mob. Comput.,* vol. 2022, Art. no. 6467866, p. 12, 2022.

[40] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Ethereum Project Yellow Paper, 2014. Accessed: 10 May 2022. [Online]. Available: https://ethereum.github.io/yellowpaper/paper.pdf.

[41] C. N. Samuel, S. Glock, F. Verdier, and P. Guitton-Ouhamou, Choice of ethereum clients for private blockchain: Assessment from proof of authority perspective, in *Proc. IEEE Int. Conf. Blockchain Cryptocurrency (ICBC),* May 2021, pp. 1–5.

[42] Y. Sompolinsky and A. Zohar, Secure high-rate transaction processing in bitcoin, in *Proc. Int. Conf. Financial Cryptogr. Data Secur. (FC 2015),* San Juan, PR, USA, Jan. 2015, pp. 507527.

[43] D. Johnson, A. Menezes, and S. Vanstone, The elliptic curve digital signature algorithm (ECDSA), *Int. J. Inf. Secur.,* vol. 1, no. 1, pp. 3663, Aug. 2001.

[44] D. Hankerson, A. Menezes, and S. Vanstone, *Guide to elliptic curve cryptography.* New York, NY, USA: Springer-Verlag, 2004.

[45] M. Qu, "SEC 2: Recommended elliptic curve domain parameters," SEC2-Ver-1.0, Certicom Res., Mississauga, ON, Canada, 1999.

[46] M. M. Islam, M. S. Hossain, M. K. Hasan, M. Shahjalal, and Y. M. Jang, FPGA implementation of high-speed area-efficient processor for elliptic curve point multiplication over prime field, *IEEE Access,* vol. 7, pp. 178811178826, Dec. 2019.

[47] M. S. Hossain and Y. Kong, "High-performance FPGA implementation of modular inversion over F_256 for elliptic curve cryptography," *IEEE Int. Conf. Data Sci. Data Intensive Syst.,* 2015, pp. 169–174.

[48] A. Menezes, Evaluation of security level of cryptography: The elliptic curve discrete logarithm problem (ECDLP), Univ. Waterloo, ON, Canada, 2001.

[49] M. M. Islam, M. K. Islam, M. Shahjalal, M. Z. Chowdhury, and Y. M. Jang, "A low-cost cross-border payment system based on auditable cryptocurrency with consortium blockchain: Joint digital currency," *IEEE Trans. Serv. Comput.*, vol. 16, no. 3, pp. 1616–1629, June 2023.
[50] R. Chaganti *et al.*, "A comprehensive review of denial of service attacks in blockchain ecosystem and open challenges," *IEEE Access*, vol. 10, pp. 96538–96555, 2022.
[51] M. Raikwar and D. Gligoroski, "DoS attacks on blockchain ecosystem," *arXiv preprint, no. 2205.13322*, May 2022. [Online]. Available: https://doi.org/10.48550/arXiv.2205.13322
[52] M. M. Merlec and H. P. In, "SC-CAAC: A smart-contract-based context-aware access control scheme for blockchain-enabled IoT systems," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 19866–19881, Jun. 1, 2024.
[53] R. Kumar, P. Kumar, R. Tripathi, G. P. Gupta, S. Garg, and M. M. Hassan, "A distributed intrusion detection system to detect DDoS attacks in blockchain-enabled IoT network," *J. Parallel Distrib. Comput.*, vol. 164, pp. 55–68, 2022.
[54] K. Scarfone and P. Hoffman, "Guidelines on firewalls and firewall policy," *NIST Special Publication 800-41*, 2009.
[55] T. Meng, Y. Zhao, K. Wolter, and C. -Z. Xu, "On consortium blockchain consistency: A queueing network model approach," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 6, pp. 1369–1382, June 2021.

Md. Mainul Islam received his B.Sc. degree in electrical and electronic engineering from Khulna University of Engineering & Technology, Khulna, Bangladesh, in April 2018. He obtained his M.Sc. degree in electronics engineering from Kookmin University, South Korea, in February 2021, achieving the Academic Excellence Award. He is currently pursuing a Ph.D. in Computer Engineering at Texas A&M University, College Station, USA. He worked as a full-time researcher at the Intelligent Blockchain Engineering Laboratory in the Department of Computer Science and Engineering, Korea University, South Korea. His research interests include blockchain, cross-border payment, central bank digital currency, cryptography, and cybersecurity. He served as a reviewer for IEEE, Elsevier, Wiley, and MDPI publishing journals.

Mpyana Mwamba Merlec received his PhD in computer science and engineering (software) in 2022, and an MSE in computer science and radio communication engineering in 2017 (with Academic Excellence Awards), both from Korea University and a BSc in computer science and engineering from the Information Systems Engineering Department from University Protestant of Lubumbashi (UPL) in 2011. He is currently a research professor at the Blockchain Research Institute, Korea University, Seoul, Korea. His primary research interests include Web 3.0, distributed computing systems, distributed ledger technologies, distributed/decentralized applications (Dapps), and the following topics: blockchain-enabled FinTech, e-commerce, supply chains, cryptocurrency algorithmic trading, cybersecurity, data privacy and security, GDPR compliance, machine learning, deep learning, Internet of things (IoT), and cyber-physical systems. He served as a reviewer for IEEE, ACM Computing Survey, Elsevier, Springer Nature, and MDPI publishing journals.

Hoh Peter IN received his B.Sc. degree in computer engineering from Korea University, South Korea, in 1990. He received his M.Sc. degree in computer science from Korea University in 1992. He pursued his Ph.D. degree in computer engineering from the University of Southern California, USA, in 1998. In 1999, he became an assistant professor at Texas A&M University, USA. He joined the Department of Computer Science at Korea University as an assistant professor in 2003 and is a professor now. He is a founder and the emeritus president of the Korean Society of Blockchain and the director of the Blockchain Research Institute, in South Korea. His main research interests are blockchain, smart contracts, and software engineering. He received the ICRE 10-Year Most Influential Paper Award in 2006. He has published over 120 research papers.